WN_MCP Plugin - Code Conventions & Architecture Guide

Project Overview

Unreal Engine 5 MCP (Model Context Protocol) plugin. UE C++ TCP Server + Python FastMCP Server (stdio) 2-layer architecture.

- **WN_MCP**: Core plugin (TCP server, command group system, Python MCP server) — **수정 금지**
- **WN_MCP_WORLD**: 실제 커맨드 구현 플러그인 (Actor, Blueprint, Component CRUD)
- **WN_MCPTest**: Example/test plugin (demonstrates how to extend WN_MCP)
- Protocol: **JSON-RPC 2.0** for all UE <-> Python communication

Terminology

Term	Meaning
Command (커맨드)	C++ side function. `FWN_MCP_CommandGroup` subclass에서 UE API를 호출하는 함수.
Tool (툴)	Python side function. `@mcp.tool` 데코레이터로 등록되어 Agent가 호출하는 함수.

- Command는 UE 엔진 내부에서 실행되며, TCP 소켓을 통해 JSON-RPC 2.0으로 응답한다.
- Tool은 Python MCP Server에서 실행되며, 필요 시 Command를 호출하여 UE와 통신한다.

Architecture

```
Agent (Claude/Cursor) <--stdio--> Python MCP Server (Tool) <--TCP JSON-RPC 2.0--> UE C++ Plugin (Command)
```

How to add functionality:

1. **C++ Command** - UE API 접근이 필요한 기능
2. `FWN_MCP_CommandGroup` 서브클래스에 커맨드 구현
3. `WN_MCP_REGISTER_COMMAND_GROUP` 매크로로 자동 등록
4. Python Tool에서 `ue_connection.execute(command_name, params, app_name=_APP)`로 호출
5. **Python Tool only** - UE 접근 불필요하거나 복잡한 오케스트레이션
6. `@mcp.tool` 데코레이터로 등록

Work Rules

1. **WN_MCP 수정 금지**: `WN_MCP/` 플러그인 코드를 수정하면 안 된다. `WN_MCP_WORLD/`만 수정 대상이다.
2. **WN_MCP Python 수정 금지**: `WN_MCP/Python/`의 Python 코드를 수정하면 안 된다. `WN_MCP_WORLD/Python/`만 수정 대상이다.
3. **사전 승인 필수**: 커맨드 이식 전 사용자에게 설명 없이 바로 구현하면 안 된다.
4. **커맨드 단위 커밋**: 커맨드 이식 후 커밋하지 않고 다음 커맨드로 넘어가면 안 된다.
5. **새 파일만 추가**: 기존 CommandGroup 파일을 수정하면 안 된다. 새 CommandGroup은 `Private/Commands/`에 새 .h/.cpp 파일로 추가한다.

C++ Command Conventions

File Structure

```
WN_MCP/Source/WN_MCP/                    # Core plugin (수정 금지)
  Public/
    WN_MCP_Module.h                      # Module header (public API, WN_MCP_API exports)
    WN_MCP_CommandGroup.h                # Base class + macros + defines
  Private/
    WN_MCP_Module.cpp                    # Module implementation
    WN_MCP_CommandGroup.cpp              # Base class implementation
    WN_MCP_ServerThread.h/.cpp           # TCP server thread

WN_MCP_WORLD/Source/
  WN_MCP_WORLD/                          # Runtime module (Engine API만 사용)
    Public/
      WN_MCP_ActorUtils.h                # 공유 유틸리티 (FindActorByName, WN_MCP_WORLD_API export)
    Private/
      WN_MCP_ActorUtils.cpp              # FindActorByName 구현
```

```
      Commands/
        WN_MCP_CommandGroup_Actor.h/.cpp         # Actor CRUD
        WN_MCP_CommandGroup_ActorComponent.h/.cpp # Actor component CRD
  WN_MCP_WORLD_Editor/                           # Editor module (UnrealEd 의존)
    Private/
      WN_MCP_BlueprintUtils.h/.cpp               # 공유 유틸리티 (FindBlueprintByName, FindSCSNodeByName)
      Commands/
        WN_MCP_CommandGroup_Blueprint.h/.cpp     # Blueprint CRUD + Spawn
        WN_MCP_CommandGroup_BlueprintComponent.h/.cpp # Blueprint component CRD
        WN_MCP_CommandGroup_Property.h/.cpp      # Property 설정
        WN_MCP_CommandGroup_Material.h/.cpp      # Material 조회/적용
        WN_MCP_CommandGroup_StaticMesh.h/.cpp    # StaticMesh 조회
        WN_MCP_CommandGroup_BlueprintVariable.h/.cpp # Blueprint 변수 추가/제거
        WN_MCP_CommandGroup_BlueprintGraph.h/.cpp    # Blueprint 그래프 노드/연결
  WN_MCP_WORLD/Python/                           # Python MCP Server
    mcp_server.py                                # FastMCP server (stdio transport)
    pyproject.toml
    tools/                                       # Tool 모듈 (용도별 분리)
      __init__.py                                # register_all_tools()
      ue_connection.py                           # UE TCP 연결 유틸리티
      actor_tools.py                             # Actor CRUD 6개
      actor_component_tools.py                   # ActorComponent CRD 3개
      blueprint_tools.py                         # Blueprint CRUD 7개 (스폰 포함)
      blueprint_component_tools.py               # BlueprintComponent CRD 3개
      property_tools.py                          # Property 설정 2개
      material_tools.py                          # Material 조회/적용 6개
      static_mesh_tools.py                       # StaticMesh 조회 2개
      blueprint_variable_tools.py                # BlueprintVariable 추가/제거 2개
      blueprint_graph_tools.py                   # BlueprintGraph 노드/연결 7개
    tests/                                       # 테스트 (unit/integration/e2e)
      conftest.py                                # 공통 fixture
      test_unit_tools.py                         # Unit 테스트 (UE Mock)
      test_integration.py                        # 통합 테스트 (UE 필요)
      test_e2e.py                                # E2E 테스트 (UE 필요)
```

Naming Rules

Category	Convention	Example
File names	`WN_MCP_` prefix + PascalCase	`WN_MCP_CommandGroup_Actor.h`
Classes (non-UObject)	`F` prefix + `WN_MCP_`	`FWN_MCP_CommandGroup_Actor`
Classes (UObject)	`U` prefix + `WN_MCP_`	`UWN_MCP_Subsystem`

Category	Convention	Example
Input params	`In` prefix	`InCommandName`, `InParamsPtr`
Output params	`InOut` prefix	`InOutCommandInfos`
Local result objects	`InResultPtr`	WN_MCP 매크로가 이 변수명에 의존
Smart pointers	`Ptr` suffix	`ListenerSocketPtr`, `InClientSocketPtr`
Booleans	`b` prefix	`bIsRunning`
Methods	PascalCase	`OnExecute`, `GetName`, `OnRegisterCommands`
Macros	`WN_` prefix + ALL_CAPS	`WN_RETURN_COMMAND_RESPONSE`
Command names	snake_case string	`"get_actors_in_level"`, `"spawn_actor"`

Section Dividers (MUST follow exactly)

```
// Top-level sections: 80 dashes
//================================================================================
// Section Name
//================================================================================

// Sub-sections inside class: 72 dashes
    //------------------------------------------------------------------------
    // Sub-section Name
    //------------------------------------------------------------------------
```

Documentation Comments

```
/// Single-line doc comment.
/// @param InName parameter description.
/// @return return value description.
```

Use `///` (triple-slash) for Doxygen. NOT `/** */` style.

Header File Template

```
#pragma once

#include "WN_MCP_CommandGroup.h"

//------------------------------------------------------------------------------
// Forward Declarations
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// FWN_MCP_CommandGroup_XXX
//------------------------------------------------------------------------------

/// Command group for XXX operations.
class FWN_MCP_CommandGroup_XXX : public FWN_MCP_CommandGroup
{
    //--------------------------------------------------------------------------
    // Overrides FWN_MCP_CommandGroup
    //--------------------------------------------------------------------------

public:

    /// Get command group name.
    FString GetName() const override;

    /// Get application name prefix for command registration.
    FString GetApplicationName() const override;

    /// Register commands.
    void OnRegisterCommands(TArray<FWN_MCP_CommandInfo>& InOutCommandInfos) const override;

    /// Execute command.
    TSharedPtr<FJsonObject> OnExecute(const FString& InCommandName, const TSharedPtr<FJsonObject>& InParamsPtr, TSh

    //--------------------------------------------------------------------------
    // Methods
    //--------------------------------------------------------------------------

public:

    /// constructor.
    FWN_MCP_CommandGroup_XXX();

    /// destructor.
```

```
    ~FWN_MCP_CommandGroup_XXX();
};
```

Source File Template

```
#include "WN_MCP_CommandGroup_XXX.h"
#include "WN_MCP_Module.h"

#include "Dom/JsonObject.h"

//------------------------------------------------------------------------
// FWN_MCP_CommandGroup_XXX
//------------------------------------------------------------------------

WN_MCP_REGISTER_COMMAND_GROUP(FWN_MCP_CommandGroup_XXX);

FWN_MCP_CommandGroup_XXX::FWN_MCP_CommandGroup_XXX()
{

}

FWN_MCP_CommandGroup_XXX::~FWN_MCP_CommandGroup_XXX()
{
}

FString FWN_MCP_CommandGroup_XXX::GetName() const
{
    return FString("XXX");
}

FString FWN_MCP_CommandGroup_XXX::GetApplicationName() const
{
    return FString("WN_MCP_WORLD");
}

void FWN_MCP_CommandGroup_XXX::OnRegisterCommands(TArray<FWN_MCP_CommandInfo>& InOutCommandInfos) const
{
    WN_MCP_REGISTER_COMMAND(InOutCommandInfos, "command_name_here", "Description of the command");
}

TSharedPtr<FJsonObject> FWN_MCP_CommandGroup_XXX::OnExecute(const FString& InCommandName, const TSharedPtr<FJsonOb
{
    //------------------------------------------------------------------------
    // command_name_here
```

```
    //------------------------------------------------------------------------

    if (InCommandName == FString(TEXT("command_name_here")))
    {
        // Implementation here

        TSharedPtr<FJsonObject> InResultPtr = MakeShareable(new FJsonObject());
        InResultPtr->SetStringField(TEXT("key"), TEXT("value"));
        WN_RETURN_COMMAND_RESPONSE(InResultPtr);
    }

    WN_RETURN_COMMAND_ERROR_TEXT_RESPONSE(WN_JSON_RPC_ERROR_METHOD_NOT_FOUND, TEXT(""));
}
```

Command Registration

Commands are registered via `FWN_MCP_CommandInfo` struct and helper macros:

```
/// Struct holding command metadata for registration.
struct FWN_MCP_CommandInfo
{
    FString Name;
    FString Description;
    bool bIsSimple;
};

/// Register a standard command (requires manual JSON response building).
WN_MCP_REGISTER_COMMAND(InOutList, "command_name", "Description")

/// Register a simple command (auto-wrapped by the framework).
WN_MCP_REGISTER_SIMPLE_COMMAND(InOutList, "command_name", "Description")
```

Each CommandGroup MUST override `GetApplicationName()` to return its plugin name (e.g. `"WN_MCP_WORLD"`). This prefix is prepended to command names during registration to prevent cross-plugin collisions. `Execute()` automatically strips the prefix before calling `OnExecute()`, so if-statements inside `OnExecute` use raw names (e.g. `"spawn_actor"`, not `"WN_MCP_WORLD_spawn_actor"`).

Response Macros (MUST use these, never hand-build response JSON)

Macro	Usage
`WN_RETURN_COMMAND_RESPONSE(InResultPtr)`	Object result
`WN_RETURN_COMMAND_TEXT_RESPONSE(TEXT("msg"))`	Text result
`WN_RETURN_COMMAND_ARRAY_RESPONSE(jsonArray)`	Array result
`WN_RETURN_COMMAND_ERROR_TEXT_RESPONSE(code, TEXT("msg"))`	Error with text
`WN_RETURN_COMMAND_ERROR_RESPONSE(code, errorObj)`	Error with object

Code Style Details

- Tabs for indentation (UE standard), NOT spaces
- Opening brace on next line for functions/classes
- Opening brace on same line for `if/else/for/while`
- Access specifiers (`public:`, `private:`) with blank line after
- Empty constructor body: opening brace on next line, empty line, closing brace
- Empty destructor body: opening brace on next line, closing brace (NO empty line)
- `FString(TEXT("..."))` for command name comparisons
- `MakeShareable(new FJsonObject())` for creating JSON objects
- Always end `OnExecute` with `WN_RETURN_COMMAND_ERROR_TEXT_RESPONSE(WN_JSON_RPC_ERROR_METHOD_NOT_FOUND, TEXT(""));`
- `OnExecute` 내 각 커맨드 블록 앞에 72-dash 구분선 추가 (커맨드 이름 포함)
- 자명한 주석 금지: `// Get required parameters`, `// Return result` 등 — 코드가 이미 설명하는 내용 반복 금지
- `InResultPtr` 앞에는 빈 줄 1개 (결과 생성부와 로직 분리)
- Inline comments start with capital letter: `// Find actor by name` (NOT `// find actor by name`)
- CommandGroup 내부 헬퍼는 파일 스코프 `static` 함수로 선언 (클래스 멤버 아님)
- 여러 CommandGroup에서 공유하는 유틸리티는 `FWN_MCP_BlueprintUtils` 등 전용 유틸리티 클래스에 `static` 멤버로 선언

Extension Plugin Pattern (like WN_MCPTest)

New command plugins MUST follow this pattern: 1. Depend on `WN_MCP` module in `.build.cs` 2. In `StartupModule()`: get `FWN_MCP_Module::Get()`, call `AddUserToolDirectories()` 3. Create `Python/` directory with tool `.py` files 4. C++ CommandGroups auto-register via `WN_MCP_REGISTER_COMMAND_GROUP` macro

---

JSON-RPC 2.0 Protocol

Python -> UE (Request)

```
{
    "jsonrpc": "2.0",
    "method": "command",
    "id": "WN_MCP_WORLD_command_name",
    "params": { ... }
}
```

UE -> Python (Success Response)

```
{
    "jsonrpc": "2.0",
    "result": { ... },
    "id": "command_name"
}
```

UE -> Python (Error Response)

```
{
    "jsonrpc": "2.0",
    "error": {
        "code": -32601,
        "message": "Method not found",
        "data": "additional info"
    },
    "id": "command_name"
}
```

Standard Error Codes

Code	Name	Usage
-32700	Parse error	Invalid JSON
-32600	Invalid Request	Not a valid request

Code	Name	Usage
-32601	Method not found	Command not registered
-32602	Invalid params	Wrong parameters
-32603	Internal error	Server-side error
-32001	Command failed	Command execution failed
-32002	Invalid arguments	Invalid command arguments

Python Tool Conventions

UE Connection

Python Tool에서 C++ Command를 호출할 때는 `ue_connection.execute()` 를 사용한다:

```
from tools import ue_connection

_APP = "WN_MCP_WORLD"  # 각 tool 모듈 상단에 선언

result = ue_connection.execute("command_name", params, app_name=_APP)
```

- `app_name` 파라미터는 C++ `GetApplicationName()` 과 일치해야 한다
- `execute()` 가 내부적으로 `"{app_name}_{command_name}"` 형태로 JSON-RPC `id` 를 구성한다

Testing

- 테스트 커버리지가 80% 미만이면 안 된다
- 새 Tool/Command 추가 시 Unit 테스트 없이 코드만 추가하면 안 된다
- Unit 테스트에서 외부 의존성(UE 연결 등)을 Mock 없이 직접 호출하면 안 된다
- 코드 변경 후 `uv run python -m pytest tests/ -q` 실행 없이 작업을 완료하면 안 된다

CRITICAL RULES

1. `WN_MCP/` 플러그인 파일(C++ 및 Python 모두)을 수정하면 안 된다. `WN_MCP_WORLD/` 만 수정 대상이다
2. 기존 CommandGroup .h/.cpp 파일에 새 커맨드를 끼워 넣으면 안 된다. 새 .h/.cpp 파일로 추가해야 한다
3. `OnExecute` 를 `WN_RETURN_COMMAND_ERROR_TEXT_RESPONSE(WN_JSON_RPC_ERROR_METHOD_NOT_FOUND, TEXT(""));` 없이 끝내면 안 된다
4. `OnExecute` 내에서 GameThread 외 스레드에서 UE API를 호출하면 안 된다 (ServerThread가 이미 GameThread로 디스패치)
5. 커맨드 이름이 다른 플러그인과 충돌하면 안 된다 — 등록 전 반드시 확인
6. `WN_MCP/Python/` 의 Python Tool을 수정하면 안 된다. `WN_MCP_WORLD/Python/` 의 Tool만 작업 대상이다
7. 사용자 승인 없이 커맨드를 이식하면 안 된다
8. 커맨드 이식 후 커밋하지 않고 다음 작업으로 넘어가면 안 된다
9. Copyright header를 `.build.cs` 외 파일에 추가하면 안 된다: `// Copyright WemadeNext, Inc. All Rights Reserved.`
10. git commit 메시지에 `Co-Authored-By` 라인을 추가하면 안 된다

Command Status

Actor Commands — `WN_MCP_WORLD` (Runtime)

Command	CommandGroup	Status
`get_actors_in_level`	Actor	Done
`find_actors_by_name`	Actor	Done
`spawn_actor`	Actor	Done
`delete_actor`	Actor	Done
`set_actor_transform`	Actor	Done
`read_actor_content`	Actor	Done

Actor Component Commands — `WN_MCP_WORLD` (Runtime)

Command	CommandGroup	Status
`add_component_to_actor`	ActorComponent	Done

Command	CommandGroup	Status
`remove_component_from_actor`	ActorComponent	Done
`read_components_from_actor`	ActorComponent	Done

Blueprint Commands — `WN_MCP_WORLD_Editor` (Editor)

Command	CommandGroup	Status
`create_blueprint`	Blueprint	Done
`read_blueprint_content`	Blueprint	Done
`compile_blueprint`	Blueprint	Done
`delete_blueprint`	Blueprint	Done
`get_blueprints`	Blueprint	Done
`find_blueprints_by_name`	Blueprint	Done
`spawn_blueprint_actor`	Blueprint	Done

Blueprint Component Commands — `WN_MCP_WORLD_Editor` (Editor)

Command	CommandGroup	Status
`add_component_to_blueprint`	BlueprintComponent	Done
`remove_component_from_blueprint`	BlueprintComponent	Done
`read_components_from_blueprint`	BlueprintComponent	Done

Property Commands — `WN_MCP_WORLD_Editor` (Editor)

Command	CommandGroup	Status
`set_static_mesh_properties`	Property	Done
`set_physics_properties`	Property	Done

StaticMesh Commands — `WN_MCP_WORLD_Editor` (Editor)

Command	CommandGroup	Status
`get_static_meshes`	StaticMesh	Done
`find_static_meshes_by_name`	StaticMesh	Done

Material Commands — `WN_MCP_WORLD_Editor` (Editor)

Command	CommandGroup	Status
`get_available_materials`	Material	Done
`apply_material_to_actor`	Material	Done
`apply_material_to_blueprint`	Material	Done
`get_actor_material_info`	Material	Done
`get_blueprint_material_info`	Material	Done
`set_mesh_material_color`	Material	Done

Shared Utilities — `WN_MCP_WORLD` (Runtime)

Utility	File	Used By
`FWN_MCP_ActorUtils::FindActorByName`	`Public/WN_MCP_ActorUtils.h`	Actor, ActorComponent, Material

Shared Utilities — `WN_MCP_WORLD_Editor`

Utility	File	Used By
`FWN_MCP_BlueprintUtils::FindBlueprintByName`	`WN_MCP_BlueprintUtils.h/.cpp`	Blueprint, BlueprintComponent, Property, Material, BlueprintVariable, BlueprintGraph

Utility	File	Used By
`FWN_MCP_BlueprintUtils::FindSCSNodeByName`	`WN_MCP_BlueprintUtils.h/.cpp`	BlueprintComponent, Property, Material

Blueprint Variable Commands — `WN_MCP_WORLD_Editor` (Editor)

Command	CommandGroup	Status
`add_variable_to_blueprint`	BlueprintVariable	Done
`remove_variable_from_blueprint`	BlueprintVariable	Done

Blueprint Graph Commands — `WN_MCP_WORLD_Editor` (Editor)

Command	CommandGroup	Status
`read_blueprint_graph`	BlueprintGraph	Done
`add_event_node`	BlueprintGraph	Done
`add_function_call_node`	BlueprintGraph	Done
`add_variable_get_node`	BlueprintGraph	Done
`add_variable_set_node`	BlueprintGraph	Done
`connect_nodes`	BlueprintGraph	Done
`disconnect_nodes`	BlueprintGraph	Done

Skipped

Command	Reason
`focus_viewport`	CRUD 범위 밖
`take_screenshot`	CRUD 범위 밖